Laporan Praktikum Algoritma & Pemrograman

Semester Genap 2025/2026

SAYA MENYATAKAN BAHWA LAPORAN PRAKTIKUM INI SAYA BUAT DENGAN USAHA SENDIRI TANPA MENGGUNAKAN BANTUAN ORANG LAIN. SEMUA MATERI YANG SAYA AMBIL DARI SUMBER LAIN SUDAH SAYA CANTUMKAN SUMBERNYA DAN TELAH SAYA TULIS ULANG DENGAN BAHASA SAYA SENDIRI.

SAYA SANGGUP MENERIMA SANKSI JIKA MELAKUKAN KEGIATAN PLAGIASI, TERMASUK SANKSI TIDAK LULUS MATA KULIAH INI.

NIM	**71251230**
Nama Lengkap	**Okky Alexander**
Minggu ke / Materi	**07 / Percabangan dan Perulangan Kompleks**

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS KRISTEN DUTA WACANA
YOGYAKARTA
2026

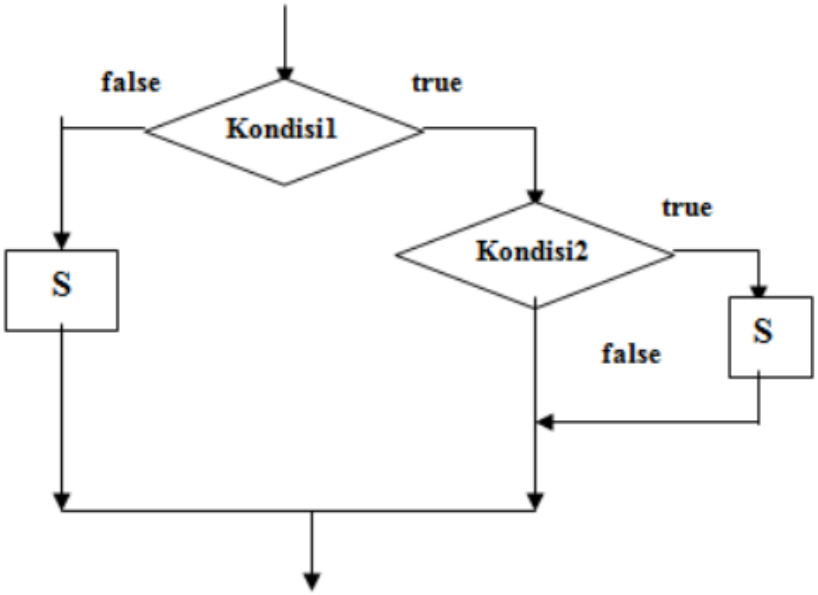
MATERI STRUKTUR PERCABANGAN KOMPLEKS

Logika pemilihan yang menyediakan banyak opsi, bukan cuma satu, di mana tiap pilihannya bisa berisi lebih dari satu tugas untuk dikerjakan. Contoh bentuk percabangan kompleks:

```
if kondisi1:
    if kondisi2:
        s
        s
else:
    s
    s
```

Gambar 6.1: Percabangan Kompleks Bentuk 1

Berikut gambar flowchartnya:



Gambar 6.2: Flowchart Percabangan Kompleks

Ada juga bentuk bentuk percabangan kompleks yang lainnya seperti:

```
if kondisi1:
    s
    if kondisi2:
        if kondisi5:
            s
        else:
            s
    elif kondisi3:

        if kondisi6:
            s

    elif kondisi4:
        s
    else:
        s
else:
    s
    s
```

Gambar 6.3: Percabangan Kompleks Bentuk 2

MATERI STRUKTUR PERULANGAN KOMPLEKS

Break

Digunakan untuk keluar dari proses looping (perulangan) secara instan. Biasanya diletakkan di dalam struktur IF agar proses berhenti tepat saat kondisi yang diinginkan tercapai.

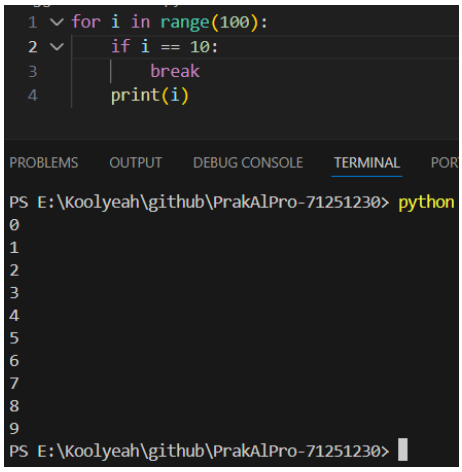
```
1  for i in range(100):
2      print(i)
3      if i == 10:
4          break
```

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS E:\Koolyeah\github\PrakAlPro-71251230> python -u
0
1
2
3
4
5
6
7
8
9
10
PS E:\Koolyeah\github\PrakAlPro-71251230>
```

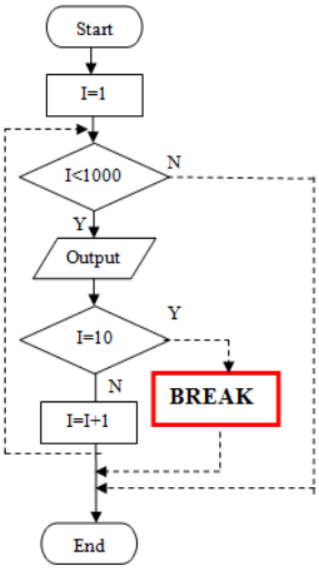
Gambar 6.4: Contoh pengunaan break dalam kode.

Program di atas akan menampilkan angka 1 sampai dengan 10 walaupun di perulangan sudah diset dari 1 sampai dengan 1000. Hal ini karena perintah break yang diberikan pada saat kondisi i=10. Angka 10 masih ditampilkan karena perintah untuk mencetak diletakkan sebelum perintah break.



Gambar 6.5: Contoh program break lain.

Meskipun batas perulangan diatur hingga angka 1000, program hanya akan mencetak angka 1 sampai 9. Hal ini terjadi karena instruksi break aktif tepat saat variabel mencapai angka 10. Karena perintah cetak berada di bawah break, angka 10 tersebut tidak sempat ditampilkan ke layar.



Gambar 6.6: Flowchart Program Menggunakan Break

Continue

Instruksi continue berfungsi untuk 'melompati' sisa perintah dalam satu iterasi dan langsung menarik program kembali ke awal perulangan. Segala baris kode yang berada di bawah continue akan diabaikan, dan proses biasanya dikendalikan oleh pernyataan IF sebagai pemicunya.

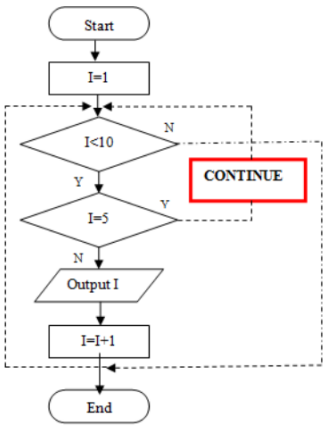
```
1  for i in range(10):
2      if i == 5:
3          continue
4      print(i)
```

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS E:\Koolyeah\github\PrakAlPro-71251230> python -u "e:\Ko
0
1
2
3
4
6
7
8
9
PS E:\Koolyeah\github\PrakAlPro-71251230>
```

Gambar 6.7: Program penggunaan continue.

Program di atas tidak menampilkan angka 5 karena pada saat angka 5 akan ditampilkan, perintah continue dijalankan, sehingga perintah mencetak di bagian bawahnya tidak akan dikerjakan dan langsung dilanjutkan ke perulangan berikutnya.



Gambar 6.8: Flowchart Program Menggunakan Continue

Perulangan Bertingkat

Perulangan kompleks, atau yang sering disebut perulangan bersarang (*nested loop*), adalah struktur di mana sebuah proses iterasi berada di dalam iterasi lainnya. Karena setiap putaran luar harus menunggu seluruh putaran dalam selesai, waktu pemrosesan cenderung meningkat. Struktur ini sangat efektif untuk menangani data yang memiliki format grid atau koordinat panjang dan lebar.

```
#Program 1
for i in range(10):
    <lakukan perintah ini>
    <lakukan perintah itu>
#Program 2
for j in range(5):
    <lakukan perintah ini>
    <lakukan perintah itu>
```

Gambar 6.9: Program perulangan.

Program 1 dan Program 2 beroperasi secara independen satu sama lain. Program 1 dirancang untuk melakukan iterasi sebanyak 10 kali, sementara Program 2 hanya menjalankan 5 kali pengulangan. Jika perulangan pada program 2 "dimasukkan" ke dalam perulangan program 1, maka menjadi:

```
for i in range(10):
    for j in range(5):
        <lakukan perintah itu di inner>
        <lakukan perintah itu di inner>
    <lakukan perintah lain di outer>
    <lakukan perintah lain lagi di outer>
```

Gambar 6.10: Program perulangan menjadi 1 program.

Dalam struktur ini, perulangan i berperan sebagai outer loop (lingkaran luar) dan perulangan j sebagai inner loop (lingkaran dalam). Mekanismenya adalah setiap kali satu tahapan pada perulangan luar berjalan, program akan menuntaskan seluruh siklus perulangan dalam sebanyak 5 kali.

Contoh perulangan dalam bentuk while:

```
while(i<=m):
    while(j<=n):
        <lakukan perintah ini di inner>
        <lakukan perintah itu di inner>
    <lakukan perintah lain di outer>
    <lakukan perintah lain lagi di outer>
```

Gambar 6.11: Program perulangan dalam bentuk while.

Bagian while i menjadi outer loop, sedangkan bagian while j menjadi inner loop. Alur pengerjaannya adalah sebagai berikut: untuk setiap 1x s/d m outer loop dijalankan akan dikerjakan n kali inner loop.

Source Github : https://github.com/tuangkeman/71251230_Okky.git

LATIHAN 7.1

```
n = int(input("input n:"))

def prima(x):
    if x < 2:
        return False
    for i in range(2, int(x**0.5) + 1):
        if x % i == 0:
            return False
    return True

for i in range(n-1, 1, -1):
    if prima(i):
        print(f"maka prima terdekat<{n} adalah {i}")
        break
```

Gambar 6.12: Program prima terdekat.

Kode ini mencari bilangan prima terbesar yang nilainya lebih kecil dari angka n yang diinputkan oleh pengguna. Secara teknis, program ini bekerja dalam dua tahap utama: pertama, mendefinisikan fungsi prima(x) yang memvalidasi apakah sebuah angka adalah bilangan prima dengan cara mengecek

pembaginya hingga akar kuadrat dari angka tersebut (x**0.5). Kedua, program menjalankan perulangan mundur (*decrement*) yang dimulai dari n-1 hingga angka 2. Begitu ditemukan angka pertama yang memenuhi kriteria prima melalui fungsi tersebut, perintah break akan segera menghentikan seluruh proses perulangan, sehingga program hanya menampilkan satu hasil yaitu bilangan prima terdekat yang posisinya berada tepat di bawah n.

LATIHAN 7.2

```
n = int(input("n = "))

for i in range(n,0,-1):
    faklah = 1
    for j in range(1,i+1):
        faklah *= j

    print(faklah,end=" ")

    for k in range(i,0,-1):
        print(k, end=" ")
    print()
```

Gambar 6.12: Program untuk menampilkan deret.

Program ini bekerja seperti mesin hitung mundur yang rapi. Untuk setiap angka dari n sampai 1, program pertama-tama menghitung total perkalian (faktorial) angka tersebut dan menampilkannya di depan. Setelah hasil perkalian muncul, program melanjutkan dengan mencetak barisan angka pembentuknya secara menurun di baris yang sama. Jadi, setiap barisnya memberikan paket lengkap: hasil akhir faktorial diikuti oleh urutan angka yang dihitung.

Contoh output:

```
n = 5
120 5 4 3 2 1
24 4 3 2 1
6 3 2 1
2 2 1
1 1
```

Gambar 6.13: Output dari program untuk menampilkan deret.

LATIHAN 7.3

```
tinggi = int(input("tinggi: "))
lebar = int(input("lebar: "))

angka = 1

for i in range(tinggi):
    for j in range(lebar):
        print(angka, end=" ")
        angka += 1
    print()
```

Gambar 6.14: Program untuk menampilkan deret.

Program ini berfungsi untuk membuat sebuah tabel angka berurutan berdasarkan tinggi dan lebar yang kamu tentukan. Cara kerjanya sangat sederhana: program akan terus menghitung maju mulai dari angka 1 dan menampilkannya baris demi baris. Setiap kali satu baris mencapai batas "lebar" yang diinput, program akan berpindah ke baris di bawahnya sampai seluruh "tinggi" terpenuhi.

```
tinggi: 5
lebar: 5
1 2 3 4 5
6 7 8 9 10
11 12 13 14 15
16 17 18 19 20
21 22 23 24 25
```

Gambar 6.15: Output dari program untuk menampilkan deret.